Module 9 — Data modelling, warehouses and ETL

Learning outcomes

Choose table grain, model facts and dimensions, understand history handling, and design a robust batch pipeline.

1. OLTP vs OLAP

OLTP supports operational transactions and current state. OLAP supports analytical scans, aggregations and historical analysis. Their schema and performance goals differ.

2. Fact and dimension tables

Fact table records measurable business events or snapshots. Dimension table describes entities such as product, customer, store or date. Declare grain before selecting columns.

3. Star and snowflake schemas

Star schema is easier for many BI queries; snowflake normalises dimension hierarchies but can require more joins. Choose based on usability and governance.

4. Slowly changing dimensions

• Type 1: overwrite old value; no history.

• Type 2: create a new version with effective start/end dates and current flag.

Use Type 2 when historical reporting must reflect attributes as they were at the event time.

5. ETL and ELT

ETL transforms before loading; ELT loads raw data then transforms in the analytical platform. Choose based on data sensitivity, latency, platform capability and cost.

6. Incremental loading

Watermark tracks last processed time/sequence. Use overlap/lookback for late updates. CDC captures inserts/updates/deletes from logs or change streams. MERGE/upsert applies changes by stable key. Backfills reprocess historical ranges.

7. Idempotency and exactly-once

A rerun should not create duplicate or inconsistent output. Stable keys, deterministic transformations and upserts make processing idempotent. Exactly-once end-to-end is difficult; at-least-once delivery plus idempotent processing is common.

8. Reliability and data quality

Monitor freshness, volume, schema, null rate, uniqueness, distribution, duration and failures. Quarantine invalid rows with reasons. Preserve raw inputs for replay subject to retention/privacy rules.

Worked design

Daily order API → raw immutable files → staging validation → dedup latest order version → curated fact/dimensions → quality/reconciliation checks → BI.

Track `run_id`, extraction time, source cursor, row counts, rejected counts and output partition.

Exercises with answer guidance

1. Model order header and order lines. Explain why one fact table at order-line grain differs from order-grain.

2. Customer changes city; historical reports need old city. Use Type 2.

3. Source sends late order update from 12 days ago. Reprocess overlapping window and MERGE by order ID/version.

4. Job fails after writing staging but before publish. Use run state/atomic publish or partition swap; do not expose incomplete output.

5. Design a reconciliation: source count/value vs target count/value by date/status; explain expected exclusions.

6. Explain why a backup is needed even with replication.

7. Explain how to handle a new optional source column vs a missing required column.

Mini design test (35 minutes, 35 marks)

Design a daily inventory snapshot model:

• declare grain;

• define dimensions and measures;

• explain late-arriving corrections;

• choose partition strategy;

• define quality checks;

• describe backfill and dashboard freshness.

External practice for this module

1. dbt Learn — complete introductory material on analytics engineering, models, tests and documentation.

2. DataTalksClub Data Engineering Zoomcamp — use the data warehouse, data ingestion and orchestration modules as a guided lab; select the relevant sections rather than trying to finish the whole programme immediately.

3. Kimball Group — dimensional modelling techniques — read about fact tables, dimensions, grain and slowly changing dimensions.

4. PostgreSQL documentation — INSERT/ON CONFLICT — practise upsert patterns for idempotent loads.

5. NYC Open Data — select a dataset and design a raw → staging → curated pipeline with quality checks.

Applied assignment: model a retail order-line fact table, customer/product/date dimensions, and a Type 2 customer dimension. Load data twice and prove the second run does not duplicate output.

Exit requirement: draw a pipeline, state table grain, describe late-arriving updates, backfills, deduplication, data quality checks and safe reruns.